

Retrieval-Augmented Generation (RAG)

The 'Frozen in Time' Problem

LLMs are trained on a fixed dataset with a knowledge cutoff date. After training, their parameters do not update — the model is 'frozen in time'. Ask it about last week's news and it will either hallucinate or admit ignorance.

RAG solves this: instead of memorising all facts in parameters, it retrieves live, verified facts at inference time and bundles them with the prompt.

How RAG Works — Step by Step

- 1. User submits a query: 'What were Apple's Q3 2025 earnings?'
- 2. Query is embedded into a vector using an embedding model.
- 3. The vector is compared against a vector database of pre-indexed documents (earnings reports, news).
- 4. Top-K most semantically similar chunks are retrieved.
- 5. Retrieved chunks + original query are bundled into a prompt: 'Given this context: [chunks], answer: [query]'.
- 6. LLM generates a grounded, up-to-date answer.

Analogy: RAG is like an open-book exam. The LLM doesn't memorise the textbook — it looks up the relevant pages at exam time.

RAG Components

Document Store: Raw documents (PDFs, web pages, databases) that are chunked and indexed.

Embedding Model: Converts text chunks into numerical vectors. Examples: OpenAI Ada, sentence-transformers.

Vector Database: Stores and searches embeddings by similarity. Examples: Pinecone, Weaviate, ChromaDB, pgvector.

LLM (Generator): Takes the retrieved context + query and generates the final response.

RAG Benefits and Limitations

Benefits	Limitations
Up-to-date information	Retrieval quality limits output quality
Source-grounded answers	Extra latency from retrieval step
Reduces hallucination	Chunking strategy is non-trivial
No retraining needed	Long documents may exceed context window

Quick-Revision Questions

- What problem does RAG solve that fine-tuning cannot?
- List the five steps in a RAG pipeline.
- Name two vector databases used in RAG systems.